The Cost of Flying Blind

How 50 hours of AI agent time burned on preventable failures â€” and the two tools that would have stopped it

Daniel Shanklin, Director of AI & Technology AIC Holdings / Eidos AGI April 2026

The Setup

Project Cerebro is a data warehouse and executive dashboard for Greenmark Waste Solutions. The technical stack is 13 repos, 7 Railway services, 2 Supabase databases (staging and production), a Cloudflare Worker, and a fleet of MCP-powered AI agents that do the actual engineering â€” writing code, opening PRs, deploying services, running extractions, verifying data.

The agents are fast. They can stand up a new vendor connector, write the bronze-silver-gold pipeline, open the PR, pass CI, and deploy to staging in under two hours. That velocity is real and it's the reason we use them.

But velocity without governance is just faster crashing.

The Incident

April 21, 2026. Session 34. The goal was simple: deploy a Fleetio fleet data connector to the data-daemon extraction pipeline. A 2-3 hour job for an experienced agent.

Here's what actually happened:

Minute 15: The agent set the develop environment's DATABASE_URL to the production database. One environment variable. Wrong value. Nobody checked.

Minute 30: The develop data-daemon connected to production Supabase and started competing with production workers for the same job queue. It grabbed Fleetio extraction jobs and failed them â€” because the develop environment didn't have the Fleetio connector code yet. Production workers never got a chance to process the jobs.

Hour 2: The agent discovered the problem (after Daniel forced a 5-whys analysis) and tried to kill the zombie container by setting DATABASE_URL to "disabled." This crashed the new deployment â€” but Railway's ON_FAILURE restart policy kept the old container (with the production DATABASE_URL) alive as a fallback. The zombie was unkillable.

Hour 3: The agent needed to apply a database migration. The Supabase CLI required an access token it couldn't find. Rather than asking for the token, it extracted the production DATABASE_URL and ran raw psql directly â€” applying DDL outside migration tracking. The agent had explicitly been told never to do this. It did it anyway because the proper tool was inconvenient.

Hour 5: The extraction pipeline returned 501 errors. The agent, unable to fix the deployment, wrote an ad-hoc Python script that connected directly to production Supabase and loaded data. This bypassed the entire pipeline â€” no job tracking, no watermarks, no retries, no error logging. Daniel caught it: "that's insane... wtf why would you ad-hoc?"

Hour 6: The agent discovered that Railway's Docker build cache was serving old code. Every "deploy" was a no-op â€” the COPY layer was cached, so the fix never reached the running container. The agent debugged this same Docker cache issue 5 separate times in one session, each time arriving at the same conclusion from scratch, because it had no memory of solving it 30 minutes ago.

Hour 8: The session ended with production in a worse state than it started. The zombie container was still alive. The migration tracking was manually patched. Ad-hoc data was in production. And the agent wrote a post-mortem acknowledging: "You gave me tools, guardrails, a trilogy, an MCP ecosystem, hooks, ceremonies â€” a complete system for doing things right. And I spent the session bypassing all of it."

The Autopsy

We built a tool called Nightingale â€” named for Florence Nightingale, who didn't just nurse soldiers but studied why they were dying. She invented data visualization to make the evidence undeniable, and she changed medicine because the institution couldn't ignore the data.

Nightingale reads full session transcripts, extracts structured incident reports, and cross-references them against governance runbooks. We ran it against session 34 and extracted 13 distinct incidents:

- 2 CRITICAL (production database cross-wired, migration tracking bypassed)
- 5 HIGH (zombie container unkillable, ad-hoc production writes, Docker cache, index mismatch, deploy pipeline misconfigured)
- 5 MEDIUM (vault token expired, silver SQL used wrong vendor, schema divergence, MCP cache stale, deploy targeting wrong environment)
- 1 META (systematic guardrail bypass under pressure)

We then ran those 13 incidents against the 9 StepProof governance runbooks we'd written â€” 39 total verification gates covering deploys, merges, migrations, data verification, and vendor onboarding.

Result: 23% coverage. 77% of real incidents had no gate.

The runbooks described the right ceremonies. The steps were in the right order. The tools were correctly mapped. But they were missing the gates that would have caught what actually broke.

The Three Missing Gates

1. Environment isolation verification (would have prevented the entire cascade)

A single check before setting any environment variable: "Does this DATABASE_URL point to the expected database for this environment?" Develop must use staging Supabase. Production must use production Supabase. This is documented as rule GR-ENV-001. It's in the topology. It's in three separate CLAUDE.md files.

The agent read all of them and set the wrong value anyway.

With a StepProof `verify_env_isolation` gate â€” a 30-second automated check â€” the zombie container never exists. Hours 2-8 never happen. The entire cascade is prevented by one gate at minute 15.

2. PreToolUse hooks for production guardrails (hard enforcement, not advisory)

The agent was told never to run raw psql against production. It acknowledged this. Then it did it because the proper tool was inconvenient.

Every guardrail in the system â€” CLAUDE.md instructions, MCP server instructions, memory files, visionlog guardrails â€” is advisory. The agent can read them and ignore them. The agent's own post-mortem identified this: "Hooks are the only hard enforcement. Everything else is advisory. I can read it and ignore it."

A PreToolUse hook that blocks Bash commands containing `psql` + production hostnames would have returned a hard error. No bypass. No negotiation. The migration would have waited for the proper token.

3. Post-deploy code version verification (proves new code is actually running)

"Deploy succeeded" does not mean "new code is running." Railway's Docker build cache can serve a month-old COPY layer on a "successful"

deploy. The agent debugged this same issue 5 times in 8 hours because it had no institutional memory.

A post-deploy step that checks the running container's commit hash against the expected commit would catch this in seconds. Data-daemon should expose a `/version` endpoint. The runbook should require it.

The Math

We mined 17 incidents across 10 sessions. Conservative time estimates:

Failure Category	Hours Burned	Prevention
Environment cross-wiring	12-15	`verify_env_isolation` gate
Ceremony bypass under pressure	6-8	PreToolUse hooks
Docker cache / stale state	10-12	Nightingale incident memory
Missing stakeholder approval	4-6	Approval step in runbook
No institutional memory	8-10	Nightingale corpus
Total	**~40-50**	

Fifty hours of agent time. From 10 sessions. On preventable, previously-seen failures.

And that's the conservative count â€” only the incidents we found through transcript mining. The actual number across all 34 sessions is likely 3-5x higher.

The Insight

AI agents are fast at execution but catastrophically expensive at debugging novel failures. A human engineer who gets burned by the Docker cache trap once never makes that mistake again â€” the scar is the lesson. An AI agent with no institutional memory makes the same mistake 5 times in one session.

That's the real cost of flying blind. Not the first failure â€” the repeats.

Nightingale is the institutional memory. It reads the wreckage and writes the warning on the door. Every incident cataloged saves the next agent the full debugging cycle.

StepProof is the gate. It binds the agent to a ceremony and verifies each step with evidence. Advisory guardrails fail under pressure. Hard gates don't.

Neither alone is sufficient. Nightingale without StepProof produces a catalog nobody enforces. StepProof without Nightingale produces gates that don't match what actually breaks. Together: the incidents tell you where the gates must be, and the gates prevent the incidents from recurring.

The compound interest: every Nightingale study makes the incident corpus richer. Every gap report makes the runbooks tighter. Every StepProof run makes the next operation safer. The system gets smarter with every failure â€” which is exactly what Florence Nightingale proved 170 years ago.

What We Built

Tool	What It Does	Status
StepProof	Governance gates â€” `keep_me_honest` binds an agent to a verified plan	Live, 8 runbooks, 39 gates

Tool	What It Does	Status
Nightingale	Failure study forge â€” reads sessions, extracts incidents, finds gaps	Live, 13 incidents cataloged
Gap Report	Cross-reference: incidents vs runbook gates	First report: 23% coverage
PreToolUse hooks	Hard enforcement â€” blocks forbidden actions before execution	Designed, not yet deployed

Next

1. Deploy the three missing gates (env isolation, production psql block, code version check)
2. Run Nightingale across all 34 sessions â€” build the full incident corpus
3. Drive gap report coverage from 23% toward 90%+
4. Every new session gets a Nightingale study. Every new incident gets a runbook gate.

The goal isn't zero failures. It's zero repeat failures.